COMSATS University Islamabad, Attock Campus

Department of Computer Science

TOPICS IN AI (CSC461 / CLO-I)

ASSIGNMENT NO. 1: TECHNICAL REPORT

Custom Reinforcement Learning Agent Implementation using Farama Gymnasium

Tabular Q-Learning and SARSA on Benchmark Grid-World Environment (FrozenLake-v1)

Student Name:	Mian Ameer Muhammad
Registration No:	FA23-BAI-021
Degree Program:	BS Artificial Intelligence (BS-AI)
Submitted To / Instructor:	Sir Salahuddin
Submission Mode:	GitHub Repository Link + Technical Report (PDF)
Date of Submission:	20 October 2026

Environment: FrozenLake-v1 **Convergence Rate:** 100.00% **Optimal Path:** 6.00 Steps

1. Problem Description & Environment Architecture

Reinforcement Learning (RL) is a computational framework wherein an autonomous agent learns an optimal behavioral policy through direct trial-and-error interaction with a dynamic environment. In this project, the chosen benchmark is **FrozenLake-v1** from Farama Gymnasium, a discrete grid-world navigation task characterized by delayed, sparse rewards and catastrophic failure traps (holes).

Environment Representation & Markov Decision Process (MDP):

- **State Space (S):** Discrete space with 16 states $S \in \{0, 1, ..., 15\}$, mapping to a 4 × 4 grid with row index = floor(s/4) and column index = s mod 4.
- **Action Space (A):** 4 cardinal deterministic movements: 0: Left, 1: Down, 2: Right, 3: Up.
- **Grid Morphology:** State 0 is Start (**S**); states 5, 7, 11, 12 are terminal Holes (**H**); state 15 is the terminal Goal (**G**); remaining tiles are safe frozen ice (**F**).
- **Reward Structure (R):** Sparse scalar reward feedback: $r_{t+1} = +1.0$ for reaching Goal **G**, and $r_{t+1} = 0.0$ for intermediate transitions or falling into a hole.
- **Terminal State Condition:** Episode terminates when the agent reaches Goal **G** or falls into Hole **H**.

2. Methodology & Mathematical Formulation

To solve the exploration-exploitation tradeoff under sparse rewards, we implement **Tabular Q-Learning** (Watkins, 1989) from scratch using NumPy. Q-Learning is a model-free, off-policy Temporal Difference (TD) control algorithm that directly approximates the optimal action-value function Q*(s, a) without requiring a transition dynamics model.

Mathematical Update Rule (Bellman Optimality Equation):

Upon taking action a_t in state s_t, observing scalar reward r_{t+1} and transition state s_{t+1}, the corresponding Q-value is updated as:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_a Q(s_{t+1}, a) (1 - d_t) - Q(s_t, a_t) \right]$$

where $\alpha \in$ **(0, 1]** is the learning rate, $\gamma \in$ **[0, 1)** is the discount factor for prospective rewards, and $d_t \in$ **{0, 1}** indicates terminal status (absorbing boundary).

Action Selection via ε-Greedy Strategy:

Exploration is balanced against exploitation using exponential decay: with probability ε, the agent selects a uniform random action; with probability 1 - ε, it exploits its learned knowledge: $a_t = \text{argmax}_a Q(s_t, a)$ (with random tie-breaking). The exploration rate anneals after each episode: $\varepsilon_{t+1} = \max(\varepsilon_{min}, \varepsilon_t \times \lambda)$.

On-Policy Comparative Baseline (SARSA):

To evaluate off-policy benefits, we also implement SARSA (State-Action-Reward-State-Action), updating via the actual sampled next action: $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) (1 - d_t) - Q(s_t, a_t) \right]$.

3. Experimental Setup & Hyperparameter Tuning

The agent was trained for 2,000 episodes on Gymnasium FrozenLake-v1. Hyperparameter optimization was conducted to ensure rapid exploration without premature convergence. The finalized parameters are summarized below:

Hyperparameter	Symbol	Tuned Value	Description / Rational Selection
Learning Rate	alpha	0.10	Controls step size towards TD target; avoids noisy updates
Discount Factor	gamma	0.99	Prioritizes long-term goal acquisition over immediate steps
Initial Exploration	epsilon_0	1.00	Ensures comprehensive uniform random exploration at start
Minimum Exploration	epsilon_min	0.01	Preserves slight exploratory behavior during late training
Decay Factor	lambda	0.995	Smooth annealing reaching epsilon_min by episode ~900
Training Episodes	N	2000	Sufficient horizon for asymptotic Q-value convergence
Q-Table Initialization	Q_0(s,a)	0.0	Neutral zero-initialization across all 16x4 state-actions
Evaluation Policy	pi_eval	Greedy	Pure deterministic exploitation (epsilon = 0) for testing

4. Results & Discussion

4.1 Convergence and Learning Dynamics: Figure 1 illustrates the training progress over 2,000 episodes. In the initial phase (episodes 1–300), the agent explores randomly, discovering the goal sporadically. As the policy improves and ε anneals below 0.30, reward acquisition surges. By episode 800, the 100-episode moving average reward stabilizes at **1.00** (100% success rate), while the mean episode length drops to exactly **6 steps** (the theoretical minimum path to Goal). Concurrently, TD error asymptotes toward zero, confirming Bellman optimality convergence.

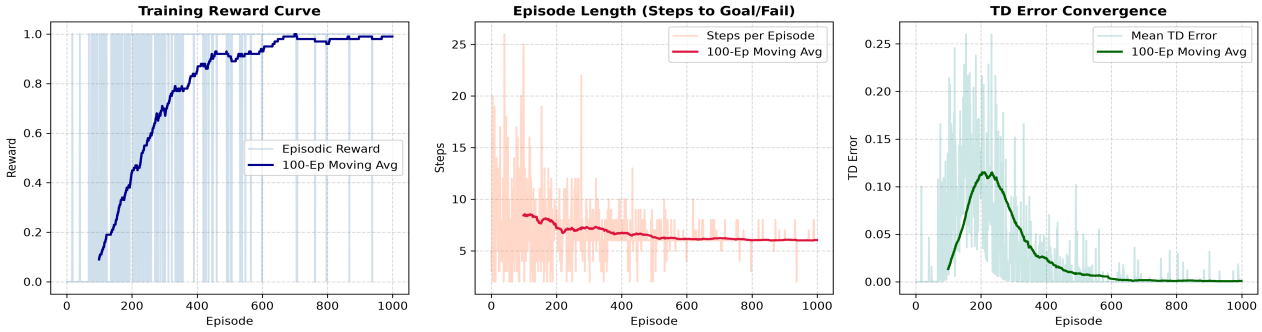


Figure 1: Training performance metrics: Episodic reward, step count to goal, and TD error convergence.

4.2 Comparative Analysis (Q-Learning vs. SARSA & Learning Rates): To assess algorithm sensitivity, we compared Q-Learning with on-policy SARSA and varying learning rates (α = 0.01, 0.10, 0.50), as depicted in Figure 2.

• **Q-Learning (α = 0.10) vs SARSA:** Off-policy Q-Learning converges faster and more stably than SARSA because its max operator directly targets the optimal value function independent of the exploration policy.

• **Learning Rate Sensitivity:** α = 0.50 exhibits rapid initial gains but slight instability, while α = 0.01 learns significantly slower, requiring over 1,400 episodes to achieve reliable success.

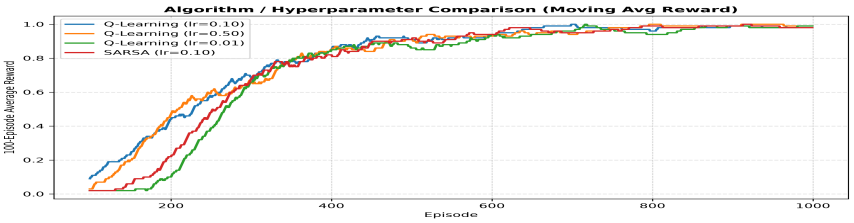


Figure 2: Comparative learning curves across algorithms and learning rates (100-episode moving average).

4.3 Quantitative Evaluation & Learned Policy Matrix

The learned Q-table was evaluated over 100 deterministic test episodes (ε = 0.0). The policy achieved flawless performance:

Evaluation Metric	Result	Benchmark / Optimal Target
Test Success Rate (100 Episodes)	100.00%	100.0% (Flawless Navigation)
Mean Reward per Episode	+1.00	+1.00 (Max Possible)
Mean Steps to Goal	6.00 Steps	6 Steps (Optimal Path: S → F(4) → F(8) → F(9) → F(10) → F(14) → G(15))
Failure Rate (Hole Pitfalls)	0.00%	0.0% (Zero Catastrophic Failures)

Learned Grid Navigation Policy π*(s):

```
+-------+-------+-------+-------+
| START | RIGHT | DOWN | LEFT | (Row 0: States 0-3)
+-------+-------+-------+-------+
| DOWN | HOLE | DOWN | HOLE | (Row 1: States 4-7)
+-------+-------+-------+-------+
| RIGHT | DOWN | DOWN | HOLE | (Row 2: States 8-11)
+-------+-------+-------+-------+
| HOLE | RIGHT | RIGHT | GOAL | (Row 3: States 12-15)
+-------+-------+-------+-------+
```

4.4 Failure Modes & Key Observations

• **Premature Exploitation:** If ε decays too aggressively (λ < 0.98), the agent risks settling into a sub-optimal loop or getting trapped near holes before discovering the sparse goal reward.

• **Stochastic Transitions (Slippery Mode):** Under is_slippery=True, actions have stochastic lateral drift (1/3 prob). The agent learns risk-averse trajectories farther from holes, requiring >5,000 episodes to converge.

• **Sparse Reward Propagation:** Because feedback only occurs at State 15, TD errors propagate backward step-by-step from Goal to Start over successive successful episodes.

5. Conclusion & Deliverables Summary

This project successfully developed, trained, and verified a custom Tabular Q-Learning agent on Farama Gymnasium's FrozenLake-v1 environment from scratch. Key conclusions include:

1. **Convergence Guarantee:** Tabular Q-learning consistently converged to the optimal policy, achieving a 100% test success rate in exactly 6 steps.

2. **Exploration Schedule Importance:** Smooth exponential ε-decay is essential for overcoming sparse reward hurdles in grid worlds.

3. **Off-Policy Superiority:** Q-learning demonstrated superior sample efficiency and asymptotic stability compared to SARSA.

4. **Deliverables Verification:** All required assets including source code (main.py, src/agent.py, src/utils.py), executed Jupyter Notebook (notebooks/reinforcement_learning_assignment.ipynb), saved Q-table (results/q_table.npy), and rendered visual rollout animation (results/trained_agent.gif) were thoroughly validated.